

8/7/26

Exp-8

Producer-Consumer Using Semaphores

Aim:

To implement the producer-consumer problem using semaphore variables in c and simulate synchronization between producer and consumer processes.

Code:

```
#include <stdio.h>
#include <stdlib.h>
int mutex = 1, full = 0, empty = 5, x = 0;
int wait (int s) {
    return --s;
}
int signal (int s) {
    return ++s;
}
void producer () {
    mutex = wait (mutex);
    empty = wait (empty);
    full = signal (full);
    x++;
    printf ("\n Producer produces item %d", x);
    mutex = signal (mutex);
}
void consumer () {
    mutex = wait (mutex);
```



```

full = wait(full);
empty = signal(empty);
printf("\n Consumer consumes item %d", x);
x--;
mutex = signal(mutex);
}

```

```

int main() {

```

```

    int n;

```

```

    printf("\n 1. Producer");

```

```

    printf("\n 2. Consumer");

```

```

    printf("\n 3. Exit");

```

```

    while(1) {

```

```

        printf("Enter choice: ");

```

```

        scanf("%d", &n);

```

```

        switch(n) {

```

```

            case 1:

```

```

                if (mutex == 1 && empty != 0) {

```

```

                    producer();

```

```

                } else {

```

```

                    printf("\n Buffer full");

```

```

                    break;

```

```

            case 2:

```

```

                if (mutex == 1 && full != 0) {

```

```

                    consumer();

```

```

                }

```

```

            else {

```

```

                printf("\n Buffer empty");

```

```

            }
            break;

```


case 3:

```
printf("Exiting...");
```

```
printf("Final items produced: %d\n", x);
```

```
exit(0);
```

default:

```
printf("Invalid choice.");
```

```
}
```

```
}
```

```
return 0;
```

```
}
```

Output:

----- Producer Consumer Problem -----

1. Producer
2. Consumer
3. Exit

Enter choice: 1

Producer produces item 1

Enter choice: 2

Consumer consumes item 1

Enter choice: 3

Exiting

Final items produced: 0

Result:

Thus, the program successfully demonstrates mutual exclusion and proper coordination for the problem.